

Cours Développement Web

Niveau Intermédiaire • Semaine 5 – Activité n°5

Application complète : CRUD PHP/MySQL, sessions, validation et médias

Durée : 06 heures | Modalité : individuel | Remise : ZIP

Intitulé	Application complète : CRUD PHP/MySQL, sessions, validation et médias
Objectifs	Maîtriser les briques nécessaires (architecture simple, validation serveur/client, sessions, upload) puis réaliser un CRUD complet sécurisé avec PDO.
Ressources	Supports du module + exercices ci-dessous + documentation PHP (PDO, sessions) et MySQL.
Type	Activité individuelle.
Outils	Laragon (PHP 8.x + MySQL + phpMyAdmin), VS Code, navigateur, DevTools.
Résultat attendu	Un fichier .zip contenant : Volet A (atelier guidé + corrections) + Volet B (mini-application CRUD réalisée par l'apprenant) + README.
Durée / Évaluation	Durée conseillée : 06 à 08 heures. Évaluation par le tuteur.

1. Présentation

Cette activité inclut des commentaires dans les extraits de code et des explications courtes après chaque étape. Objectif : aligner l'atelier sur les notions vues en cours (validation, sessions, upload) tout en gardant un code propre et réutilisable. Le code utilisé peut vous donner une autre alternative pour écrire le code PHP.

2. Volet A — Atelier guidé (avec solutions + explications)

Nous construisons un mini-CRUD 'Articles' (titre, contenu, image) afin de pratiquer :

- validation serveur (réutilisable),
- sessions (messages flash, protection),

- upload (contrôle MIME/taille + renommage),
- CRUD complet (Create/Read/Update/Delete) avec PDO et requêtes préparées.

2.1 Mise en place

1. Démarrer Laragon (Start).
2. Créer : C:\laragon\www\crud_articles/
3. Créer l'arborescence ci-dessous.
4. Ouvrir : http://localhost/crud_articles/public/

Arborescence (rappel)

```
crud_articles/  
  public/  
    index.php  
    create.php  
    edit.php  
    delete.php  
    login.php  
    logout.php  
    assets/  
      uploads/  
      app.js  
      style.css  
  app/  
    config.php  
    db.php  
    helpers.php  
    validations.php  
    auth.php  
    helpers_upload.php  
  sql/  
    schema.sql  
    donnees.sql  
  README.txt
```

Pourquoi cette structure : On sépare le code réutilisable (app/) des pages accessibles au navigateur (public/). Cela évite de dupliquer la connexion DB, les validations et les helpers dans chaque page. Nous remarquons ici une autre structure possible pour le projet. Nous avons utilisé dans les ressources de cette semaine une structure MVC.

2.2 Étape 1 — Base de données

On crée une base simple : users (authentification) + articles (contenu géré par le CRUD). L'upload stocke le chemin du fichier dans articles.image_path.

Solution — schema.sql

```
-- sql/schema.sql
-- 1) Base de données
CREATE DATABASE IF NOT EXISTS crud_app
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE crud_app;

-- 2) Table users : pour login/logout
-- email unique + password_hash (hash sécurisé)
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3) Table articles : ressource CRUD
CREATE TABLE IF NOT EXISTS articles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titre VARCHAR(120) NOT NULL,
  contenu TEXT NOT NULL,
  image_path VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NULL DEFAULT NULL
);
```

À retenir : VARCHAR pour les champs courts (titre/email), TEXT pour les contenus longs, UNIQUE sur email, et champs created_at/updated_at pour tracer les modifications.

Solution — donnees.sql

donnees.sql ajoute un utilisateur et quelques articles. Le mot de passe n'est jamais stocké en clair : on stocke un hash. En cours, vous pouvez générer un hash avec password_hash('admin123', PASSWORD_DEFAULT).

```
-- sql/donnees.sql
USE crud_app;

-- Utilisateur de test
-- Remarque : le hash peut varier d'une machine à l'autre, c'est normal.
INSERT INTO users (email, password_hash) VALUES
('admin@demo.tn', '$2y$10$e0NRx0Qq3r8z0Zb5z5VgDeH4K2Zk7Bq1r0Yc5xkqvY3b6xJq9vYgS');

-- Articles de test
INSERT INTO articles (titre, contenu, image_path) VALUES
('Premier article', 'Contenu de démonstration.', NULL),
('Deuxième article', 'Un autre contenu.', NULL);
```

2.3 Étape 2 — Configuration + connexion PDO

PDO est recommandé car il gère les requêtes préparées et une API uniforme. Nous configurons : `ERRMODE_EXCEPTION` (pour voir les erreurs), `FETCH_ASSOC` (tableaux associatifs).

Solution — app/config.php

```
<?php
// app/config.php
// Objectif : centraliser les paramètres de connexion.
return [
    'db' => [
        'host' => '127.0.0.1',
        'name' => 'crud_app',
        'user' => 'root',
        'pass' => '',
        'charset' => 'utf8mb4'
    ]
];
```

Solution — app/db.php

```
<?php
// app/db.php
// Objectif : créer UNE connexion PDO réutilisable dans toutes les pages.

$config = require __DIR__ . '/config.php';
$db = $config['db'];

$dsn = "mysql:host={$db['host']};dbname={$db['name']};charset={$db['charset']}";

$options = [
    // Affiche les erreurs sous forme d'exceptions (utile en dev)
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    // fetch() renvoie un tableau associatif
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    // Désactive l'émulation pour utiliser les prepares natifs (plus sûr)
    PDO::ATTR_EMULATE_PREPARES => false,
];

try {
    $pdo = new PDO($dsn, $db['user'], $db['pass'], $options);
} catch (PDOException $e) {
    // En production : ne pas afficher $e->getMessage()
    die('Erreur connexion DB : ' . $e->getMessage());
}

return $pdo;
```

Lien avec le cours : Le 'DSN' décrit le pilote (mysql) et la base. La connexion est créée une seule fois et importée dans chaque page via require.

2.4 Étape 3 — Sessions + messages flash + échappement

Les sessions servent à : (1) mémoriser l'utilisateur connecté, (2) afficher des messages 'flash' après redirection (ex : 'Article ajouté'). On crée aussi e() pour échapper l'affichage et éviter les injections HTML (XSS).

Solution — app/helpers.php

```
<?php
// app/helpers.php

// 1) Échapper toute sortie HTML
function e($v){
    return htmlspecialchars((string)$v, ENT_QUOTES, 'UTF-8');
}

// 2) Démarrer la session si nécessaire
function start_session(){
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }
}

// 3) Flash message : stocké en session et affiché UNE fois
function set_flash($type, $message){
    start_session();
    $_SESSION['flash'] = ['type'=>$type, 'message'=>$message];
}

function get_flash(){
    start_session();
    if (!isset($_SESSION['flash'])) return null;
    $f = $_SESSION['flash'];
    unset($_SESSION['flash']);
    return $f;
}
```

Pourquoi e() est obligatoire : Même si vos données viennent de la base, elles peuvent contenir des caractères spéciaux. e() empêche qu'un contenu utilisateur injecte du HTML ou du script.

2.5 Étape 4 — Validation serveur réutilisable

Plutôt que de répéter des if(...) partout, on met la validation dans une fonction. Elle renvoie : (1) un tableau d'erreurs, (2) des valeurs nettoyées.

Solution — app/validations.php

```
<?php
// app/validations.php

function validate_article($data, $file){
    $errors = [];

    // Nettoyage : trim() supprime les espaces inutiles
    $titre = trim($data['titre'] ?? '');
    $contenu = trim($data['contenu'] ?? '');
}
```

```

// Règles vues en cours : présence + longueur
if ($titre === '' || mb_strlen($titre) < 3 || mb_strlen($titre) > 120) {
    $errors['titre'] = 'Titre invalide (3 à 120 caractères).';
}

if ($contenu === '' || mb_strlen($contenu) < 10) {
    $errors['contenu'] = 'Contenu invalide (min 10 caractères).';
}

// Upload : contrôle taille + type MIME
if ($file && ($file['error'] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_NO_FILE) {
    if ($file['error'] !== UPLOAD_ERR_OK) {
        $errors['image'] = 'Erreur lors de l'upload.';
    } else {
        $max = 2 * 1024 * 1024; // 2MB
        if ($file['size'] > $max) $errors['image'] = 'Image trop grande (max 2MB).';

        // mime_content_type() donne le type réel du fichier
        $allowed = ['image/jpeg', 'image/png', 'image/webp'];
        $mime = mime_content_type($file['tmp_name']);
        if (!in_array($mime, $allowed, true)) $errors['image'] = 'Format image non autorisé.';
    }
}

return [$errors, ['titre'=>$titre, 'contenu'=>$contenu]];
}

```

2.6 Étape 5 — Upload (renommage + stockage)

On renomme le fichier uploadé pour éviter les collisions (deux fichiers 'image.png'). On stocke seulement le chemin en base, pas le fichier lui-même.

Solution — app/helpers_upload.php

```

<?php
// app/helpers_upload.php

function upload_image($file){
    // Extension d'origine (ex: png)
    $ext = pathinfo($file['name'], PATHINFO_EXTENSION);

    // Nom unique : date + aléatoire
    $new = 'img_' . date('Ymd_His') . '_' . bin2hex(random_bytes(4)) . '.' . $ext;

    // Dossier de destination (côté serveur)
    $destDir = __DIR__ . '/../public/assets/uploads/';
    $destPath = $destDir . $new;

    if (!is_dir($destDir)) mkdir($destDir, 0777, true);

    // move_uploaded_file() déplace le fichier temporaire vers le dossier final
    if (!move_uploaded_file($file['tmp_name'], $destPath)) {
        return null;
    }

    // Chemin relatif depuis public/

```

```

    return 'assets/uploads/' . $new;
}

```

2.7 Étape 6 — Create : formulaire + validation + INSERT préparé

Create suit toujours le même schéma :

- 1) si GET : afficher le formulaire ;
- 2) si POST : valider, puis INSERT préparé ;
- 3) redirection + message flash (PRG : Post/Redirect/Get).

Solution — public/create.php

```

<?php
require_once __DIR__ . '/../app/helpers.php';
require_once __DIR__ . '/../app/validations.php';
$pdo = require __DIR__ . '/../app/db.php';
start_session();

$errors = [];
$values = ['titre'=>'', 'contenu'=>''];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1) Validation
    [$errors, $values] = validate_article($_POST, $_FILES['image'] ?? null);

    if (empty($errors)) {
        // 2) Upload (si présent)
        $imgPath = null;
        if (isset($_FILES['image']) && $_FILES['image']['error'] !== UPLOAD_ERR_NO_FILE) {
            require_once __DIR__ . '/../app/helpers_upload.php';
            $imgPath = upload_image($_FILES['image']);
        }

        // 3) INSERT préparé (anti-injection SQL)
        $stmt = $pdo->prepare('INSERT INTO articles (titre, contenu, image_path) VALUES (:t, :c, :i)');
        $stmt->execute([':t'=>$values['titre'], ':c'=>$values['contenu'], ':i'=>$imgPath]);

        // 4) Message flash + redirection
        set_flash('ok', 'Article ajouté avec succès.');
```

header('Location: index.php');

```

        exit;
    }
}
?>
<!doctype html>
<html lang="fr">
<head>
    <meta charset="utf-8"><meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Créer</title>
    <link rel="stylesheet" href="assets/style.css">
    <script src="assets/app.js" defer></script>
</head>
<body>
<div class="container">
    <h1>Créer un article</h1>

    <?php if (!empty($errors)): ?>
        <div class="alert error">Veuillez corriger les erreurs ci-dessous.</div>
    <?php endif; ?>

```

```

<form method="post" enctype="multipart/form-data">
  <label>Titre</label>
  <input name="titre" value="<?= e($values['titre']) ?>">
  <?php if(isset($errors['titre'])) echo ' <small
style="color:#b00020">'.e($errors['titre']).'</small>'; ?>

  <label>Contenu</label>
  <textarea name="contenu"><?= e($values['contenu']) ?></textarea>
  <?php if(isset($errors['contenu'])) echo ' <small
style="color:#b00020">'.e($errors['contenu']).'</small>'; ?>

  <label>Image (optionnel)</label>
  <input type="file" name="image" accept="image/*">
  <?php if(isset($errors['image'])) echo ' <small
style="color:#b00020">'.e($errors['image']).'</small>'; ?>

  <button class="btn" type="submit">Enregistrer</button>
  <a class="btn secondary" href="index.php">Retour</a>
</form>
</div>
</body>
</html>

```

2.8 Étape 7 — Read : liste + flash messages

Principe : On liste les articles via SELECT. On affiche ensuite les liens vers edit/delete. Le message flash s’affiche après une redirection (ex : après Create/Delete).

Solution — public/index.php

```

<?php
require_once __DIR__ . '/../app/helpers.php';
$pdo = require __DIR__ . '/../app/db.php';
start_session();

$flash = get_flash();
$rows = $pdo->query('SELECT id, titre, created_at FROM articles ORDER BY id DESC')->fetchAll();
?>
<!doctype html>
<html lang="fr">
<head><meta charset="utf-8"><title>Articles</title>
<link rel="stylesheet" href="assets/style.css"></head>
<body>
<div class="container">
  <h1>Articles</h1>

  <?php if($flash): ?>
    <div class="alert <?= e($flash['type']) ?>"><?= e($flash['message']) ?></div>
  <?php endif; ?>

  <p><a class="btn" href="create.php">+ Nouvel article</a></p>

  <ul>
    <?php foreach($rows as $r): ?>
      <li>
        <?= e($r['titre']) ?>
        - <a href="edit.php?id=<?= e($r['id']) ?>">Modifier</a>
        - <a href="delete.php?id=<?= e($r['id']) ?>">Supprimer</a>
      </li>
    </?php>
  </ul>

```



```

        <?php endforeach; ?>
    </ul>
</div>
</body>
</html>

```

2.9 Étape 8 — Update : formulaire pré-rempli + UPDATE

Update nécessite 2 étapes : (1) charger l'article par id (GET), (2) mettre à jour (POST). Important : l'id est toujours passé en paramètre et utilisé dans une requête préparée.

Solution — squelette edit.php

```

<?php
require_once __DIR__ . '/../app/helpers.php';
require_once __DIR__ . '/../app/validations.php';
$pdo = require __DIR__ . '/../app/db.php';
start_session();

$id = (int)($_GET['id'] ?? 0);

// 1) Charger l'article (GET)
$stmt = $pdo->prepare('SELECT * FROM articles WHERE id = :id');
$stmt->execute([':id'=>$id]);
$article = $stmt->fetch();

if(!$article){
    set_flash('error','Article introuvable');
    header('Location: index.php');
    exit;
}

$errors = [];
$values = ['titre'=>$article['titre'], 'contenu'=>$article['contenu']];

// 2) Traiter la mise à jour (POST)
if($_SERVER['REQUEST_METHOD']=='POST'){
    [$errors,$values] = validate_article($_POST, $_FILES['image'] ?? null);

    if(empty($errors)){
        $stmt = $pdo->prepare('UPDATE articles SET titre=:t, contenu=:c, updated_at=NOW() WHERE id=:id');
        $stmt->execute([':t'=>$values['titre'], ':c'=>$values['contenu'], ':id'=>$id]);

        set_flash('ok','Article mis à jour');
        header('Location: index.php');
        exit;
    }
}
?>
<!-- HTML : similaire à create.php, mais avec $values pré-remplies -->

```

2.10 Étape 9 — Delete : confirmation + DELETE

Delete doit inclure une confirmation (page ou boîte de dialogue). Ici, on utilise une page de confirmation. Le DELETE n'est exécuté qu'en POST (bonne pratique).

Solution — delete.php

```
<?php
require_once __DIR__ . '/../app/helpers.php';
$pdo = require __DIR__ . '/../app/db.php';
start_session();

$id = (int)($_GET['id'] ?? 0);

// Exécuter la suppression uniquement si l'utilisateur confirme (POST)
if($_SERVER['REQUEST_METHOD']=='POST'){
    $stmt = $pdo->prepare('DELETE FROM articles WHERE id=:id');
    $stmt->execute([':id'=>$id]);

    set_flash('ok','Article supprimé');
    header('Location: index.php');
    exit;
}
?>
<!doctype html>
<html lang="fr"><head><meta charset="utf-8"><title>Supprimer</title>
<link rel="stylesheet" href="assets/style.css"></head>
<body>
<div class="container">
    <h1>Confirmation suppression</h1>
    <form method="post">
        <p>Confirmer la suppression de l'article #<?= e($id) ?> ?</p>
        <button class="btn" type="submit">Oui, supprimer</button>
        <a class="btn secondary" href="index.php">Annuler</a>
    </form>
</div>
</body></html>
```

2.11 Étape 10 — Validation côté client (JS)

La validation JS améliore l'expérience (retour rapide), mais elle peut être contournée. La validation serveur reste obligatoire.

Solution — public/assets/app.js

```
// public/assets/app.js
// But : éviter un envoi vide (UX). La sécurité reste côté serveur.

document.addEventListener('DOMContentLoaded', () => {
    const form = document.querySelector('form');
    if(!form) return;

    form.addEventListener('submit', (e) => {
        const titre = form.querySelector('[name="titre"]');
        const contenu = form.querySelector('[name="contenu"]');

        if(titre && titre.value.trim().length < 3){
            alert('Titre trop court (min 3).');
            e.preventDefault();
            return;
        }

        if(contenu && contenu.value.trim().length < 10){
```

```

        alert('Contenu trop court (min 10).');
        e.preventDefault();
    }
    });
});

```

2.12 Étape 11 — Sessions : login/logout + protection simple

On protège create/edit/delete. Si l'utilisateur n'est pas connecté, on le redirige vers login.php. On stocke l'id utilisateur dans \$_SESSION['user_id'].

Solution — app/auth.php

```

<?php
require_once __DIR__ . '/helpers.php';

function require_auth(){
    start_session();

    if (!isset($_SESSION['user_id'])) {
        set_flash('error', 'Veuillez vous connecter.');
        header('Location: login.php');
        exit;
    }
}

```

Où l'utiliser : En haut de create.php, edit.php, delete.php : appeler require_auth(); Cela protège les actions sensibles.

2.13 Sécurité & qualité — checklist Volet A

- Toujours prepared statements (INSERT/UPDATE/DELETE).
- Toujours e() / htmlspecialchars() à l'affichage.
- Upload : vérifier MIME + taille, renommer.
- Delete : confirmation et exécution en POST.
- Sessions : session_start() avant usage, logout nettoie la session.
- Bonus : CSRF token sur les formulaires.

3. Volet B — Projet CRUD complet (à réaliser par l'apprenant)

Réaliser une application CRUD complète en intégrant validation serveur/client, sessions, upload et bonnes pratiques de sécurité.

3.1. Consignes :

Création de la Base de Données :

- Créez une base de données nommée "bibliotheque_db" à l'aide de PHPMyAdmin.
- Dans cette base de données, créez une table nommée "livres" avec les colonnes suivantes :
 - id (clé primaire auto-incrémentée),
 - titre (VARCHAR, 255 caractères),
 - auteur (VARCHAR, 255 caractères),
 - annee_publication (INT),
 - categorie (VARCHAR, 50 caractères),
 - image (VARCHAR, 50 caractères).

Insertion de Données :

- Insérez trois livres fictifs dans la table "livres" en utilisant PHPMyAdmin. Ajouter dans le champs image les noms de trois fichiers images, et que vous placez dans le même dossier que le code de ce script.

Interrogation de la Base de Données :

- Créez un script PHP qui se connecte à la base de données "bibliotheque_db" et sélectionne tous les livres de la catégorie "Science-fiction". Affichez les résultats dans une liste. Afficher ici simplement les noms des fichiers images
- Écrivez un autre script PHP qui sélectionne les titres des livres écrits par l'auteur "Jules Verne". Affichez ces titres dans un tableau html avec affichage des images des livres.

Mise à Jour de Données :

- Créez un script PHP pour mettre à jour l'année de publication du livre dont le titre est "Vingt mille lieues sous les mers" pour la changer en 1870. Vous devez d'abord afficher les données actuelles dans un formulaire et ajouter un bouton de mise à jour. Assurez-vous que la mise à jour est effectuée correctement après clic sur le bouton correspondant.

Suppression de Données :

- Écrivez un script PHP pour supprimer tous les livres de la catégorie "Romans historiques" de la table "livres". Assurez-vous que les livres sont supprimés de manière appropriée.

Sélection de Données Supplémentaires :

- Créez un script PHP pour sélectionner tous les livres dont l'année de publication est postérieure à 2000. Affichez ces livres avec leurs titres et leurs années de publication.

Affichage de Données Générales :

- Écrivez un script PHP pour sélectionner les titres et les auteurs de tous les livres de la table "livres". Affichez ces informations dans un tableau.

Comptage de Données :

- Créez un script PHP pour compter le nombre total de livres dans la table "livres". Affichez ce nombre.

Sélection des Premiers Livres :

- Écrivez un script PHP pour sélectionner les titres des 5 premiers livres de la table "livres" par ordre alphabétique. Affichez ces titres dans une liste.

Sélection des Catégories Distinctes :

- Créez un script PHP pour sélectionner les catégories de livres distinctes de la table "livres". Affichez ces catégories dans une liste.

3.2 Exigences minimales

1. CRUD complet (C/R/U/D) sur une ressource (contacts/produits/articles).
2. PDO + requêtes préparées.
3. Validation serveur réutilisable + validation client (JS/HTML5).
4. Sessions : login/logout + flash messages + pages protégées.
5. Upload média : type/taille + renommage + stockage chemin en BD.
6. Sécurité : escape output + confirmation suppression + intro CSRF (bonus).

4. Remise

- Un fichier .zip nommé : NOM_Prenom_S5_CRUD_Complet.zip
- README : import SQL + identifiants + URLs + fonctionnalités.